

AvalanCH: A Digital Twin of the Swiss Alps

Laura Kubler

2485956

Polytechnique Montréal

laura-marianne.kubler@etud.polymtl.ca

Kevin Peymani

2075668

Polytechnique Montréal

kevin.peymani@etud.polymtl.ca

Kevin Garnier

2489027

Polytechnique Montréal

kevin.garnier@etud.polymtl.ca

Rémy Fayet

2488983

Polytechnique Montréal

remy.fayet@etud.polymtl.ca

Abstract—The AvalanCH project aims to develop a digital twin of the Swiss Alps. In alignment with the broader European Union initiative to create a highly accurate digital twin of the Earth, this project focuses on a regional-scale digital twin of the Alps and their mountainous environment. Using publicly available data from the Swiss government, the project enables simulations that contribute to improved avalanche risk management and, more generally, enhanced mountain safety.

The project addresses practical applications such as avalanche prevention, search-and-rescue support, ski resort planning, and the sustainable management of mountain resources. For example, the digital twin could be used to more efficiently detect avalanche risks and to simulate avalanche scenarios in order to estimate the impact of avalanches on infrastructures and human activity

Index Terms—Digital Twin, Avalanche, Avalanche Prevention, Avalanche Prediction

I. INTRODUCTION

AvalanCH is conceived as an operational decision-support platform for avalanche risk anticipation and prevention in mountain terrain. The system takes environmental measurements and transforms them into more meaningful variables that can be visualized. It simulates snow state evolution, and outputs spatially explicit risk indicators that can be visualized. These outputs can be analyzed and used to simulate avalanches inside AvalanCH. The project aims to address the issue of visualizing avalanche danger dynamically in time and space. It wants to be of value for avalanche hazard management.

A. Motivation

Modern mountain hazard management can no longer rely on static or delayed reports. Because conditions like snow stability, rockfall potential, and water levels can shift in a matter of minutes, authorities require instantaneous data streams to make life-saving decisions on the fly. Generalized regional warnings are often too broad to be effective for specific infrastructure or hiking trails. There is a growing necessity for hyper-local intelligence that accounts for the unique topography, elevation, and microclimate of a single slope or valley. It is no longer enough to simply assess a

“danger level”. Decision-makers and the public need more specific informations to understand the risks involved.

This need for real-time, high-resolution environmental modeling is also reflected in the large-scale European initiative, Destination Earth [1].

AvalanCH should improve temporal responsiveness by automating pipeline steps: data retrieval to map publications. It should also improve spatial consistency by providing gridded outputs directly usable in a user-interface. It should also improve resource management by providing visualization of what-if scenarios of avalanche triggering.

The platform can already do the following. It can fetch and preprocess meteorological information, including fallback strategies for missing channels. It can run topography and environment aware simulations of snow to generate gridded snow indicators. It can extract features from snow profile outputs and feed a trained risk model that predict danger levels spatially. These can then be visualized to help make select what-if scenarios of avalanche triggering.

However, there are several things AvalanCH cannot do that would be required for a fully autonomous hazard management digital twin. It cannot execute any physical interventions in the environment (for example, automated road controls, dynamic barriers for skiers, managed avalanche release system, ...)

B. Classification

Using Kritzinger’s framework [2] as shown in Fig. 1, the current AvalanCH-DT implementation is best classified as a Digital Shadow.

The classification rationale is as follows. In a Digital Model, data exchange between physical and digital entities is largely manual and not continuously synchronized. AvalanCH-DT exceeds that level because it includes automated ingestion and processing pipelines that regularly update digital state from physical measurements. In a Digital Twin, data exchange is bidirectional and automated: changes in the physical system update the digital representation, and digital outputs can automatically influence or actuate the physical system. AvalanCH-DT does not yet satisfy this second condition. Its information flow is primarily from physical to digital,

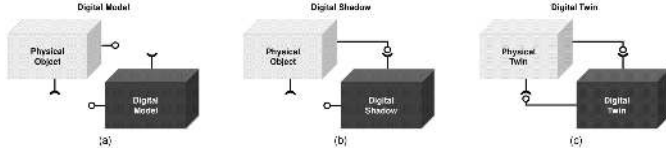


Fig. 1. Kritzinger's framework

while control actions in the field remain human-mediated and external to the platform.

This means the system already delivers substantial twin-like value in observability, simulation, and forecasting, but it does not yet implement automated, trusted, and governable actuation back into the physical domain. It is therefore more advanced than a static or manually updated digital model, yet not a full digital twin under strict bidirectional criteria. The Digital Shadow label is what fits the best our project.

To reach a true Digital Twin status, several concrete capabilities are needed.

II. ARCHITECTURE

Our Digital Twin has a set of unique challenges, listed in Table I. Because of that, the standard approach of microservices wasn't really adapted. We therefore propose this slightly modified framework we used to design our large-scale Digital Twin.

A. Microservices, async services and pipelines

The digital twin is made of interconnected units : (micro)services, async (micro)services and pipelines.

A pipeline (P) is a unit that given some inputs produces some outputs synchronously. A pipeline's outputs may only be used by a single unit.

A (micro)service (S) is a unit that given some input produces some output synchronously. A (micro)service's outputs should be used by at least 2 units. A (micro)service therefore differ from a pipeline by design : it should be reusable accross the architecture, hence its outputs should be designed as an API.

An async (micro)service (AS) is a (micro)service with a memoized output.

These units can be combined, as shown in Table II.

The distinction between service and async service is essential for us. Because we can't afford to duplicate data, nor having the DT wait for data synchronously, we sometimes need to reason in term of data produced instead of (micro)service calling. This is something we still need to

TABLE I : CHALLENGES FACED WHILE DESIGNING THE DT

Challenge	Description
Scale	The scales involved range from a few meters to several kilometers
Density	The data coverage is excellent, but this means that the DT has to crunch a large amount of data

TABLE II : COMBINATION OF UNITS

	S	AS	P
S	S	AS	S
AS	AS	AS	AS
P	S	AS	P

clarify further. This section is only intended to provide a brief overview of how we designed the DT, although it can still be considered a microservices-based DT.

III. DT SERVICES

A. Risk Prediction Service

a) Meteo Data Processing:

Data Source and Acquisition Strategy:

The avalanche risk system relies on real-time observations from the Intercantonal Measurement and Information System (IMIS), a network of automated weather stations distributed across the Swiss Alps [3] (Fig. 2). Data is fetched programmatically via HTTP API endpoints, providing continuous time-series measurements at hourly resolution:

- **Air Temperature (TA):** $^{\circ}C$, typically ranging from -20 to $+15^{\circ}C$ at high elevations
- **Relative Humidity (RH):** %, critical for radiation balance and condensation processes
- **Wind Velocity (VW):** m/s , driving snow transport and wind loading patterns
- **Wind Direction (DW):** degrees, determines aspect-specific exposure to wind-slab formation
- **Incoming Shortwave Radiation (ISWR):** W/m^2 , solar forcing crucial for surface energy balance
- **Incoming Longwave Radiation (ILWR):** W/m^2 , derived from temperature/humidity relationships or direct measurement

Data Preprocessing and Quality Control:

Data undergoes validation and gap-filling. All measurements are synchronized to a common UTC timestamp. Short gaps are interpolated linearly, bigger gaps



Fig. 2. IMIS Stations in the Zermatt municipality [3]

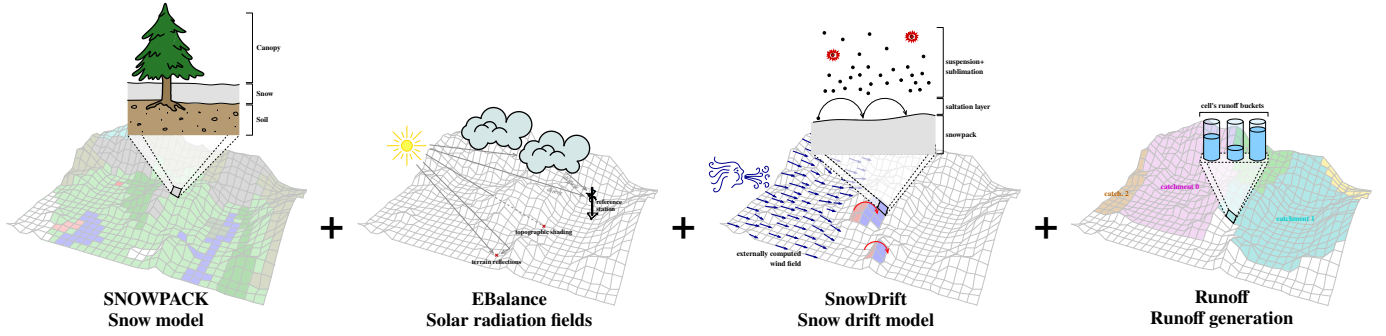


Fig. 3. Alpine 3D's four modules (from left to right): Snowpack, EBalance, SnowDrift and Runoff

trigger warnings but provide a default value for each measurement. All inputs are standardized to SI units. When Incoming Longwave Radiation is unavailable, it is derived using the Angström formula:

$$ILWR = \varepsilon \sigma T^4 - RH \text{ correction factor} \quad (1)$$

where ε is atmospheric emissivity, estimated from vapor pressure and cloud fraction.

SMET file generation:

Preprocess data is serialized into the SMET (Standard Meteo) format (Listing 1). This format is widely used by the MeteoIO/Snowpack ecosystem. A SMET file contains header metadata (location, coordinate system, timezone), column definitions (timestamp, measured/derived variables), data rows (one per hourly timestep, with missing values flagged as NaN).

b) Alpine3D:

Model Description:

Alpine3D is a spatially distributed, physics-based model that simulates snow accumulation, ablation, and surface energy balance across a Digital Elevation Model (DEM) at high resolution [4]. We used 50m cells but it can run on much smaller cells. Unlike point-based snowpack simulations, Alpine3D accounts for:

- Slope-angle-dependent radiation
- Aspect-driven temperature and wind variations
- Gravitational effects on mass balance
- Lateral snow redistribution through wind transport

Gridded Input Preparation:

Alpine3D requires (Fig. 3):

- **DEM (Digital Elevation Model):** For this we derived the one from swisstopo data, making it a 50m resolution

for computation reasons. It covers the region and computes for each cell:

- Slope angle (β , degrees): affects incoming solar radiation
- Aspect (α , degrees): determines diurnal heating patterns
- Horizon angles: self-shading by surrounding topography
- **Landuse raster:** This classifies what is on each cell (forest, grass, rock, glacier).
- **Meteorological forcing:** Alpine3D interpolates the meteorological data from the stations SMET files. How we want to interpolate the data can be described in the configuration file.

What types of file we want Alpine3D to use, the units and other parameters are all defined in one configuration file. As advised by the documentation of Alpine3D we used their graphical interface, Inishell [5], to do it. However, using Inishell turned out to be difficult and even though our file was supposed to be correct we still managed to have errors when running Alpine3D. After many tries we found a configuration file that worked and we decided to leave it as it is.

Alpine3D Runtime Configuration:

After defining the input parameters Alpine3D has runtime configuration parameters [4], they include:

- **Radiation balance:** Incoming shortwave (sun angle, albedo, cloud factor) + incoming longwave (emissivity)
- **Albedo evolution:** Fresh snow starts at 0.85, decreases with age (empirical exponential decay)
- **Wind transport model:** Mass flux of drifting snow, erosion/deposition patterns
- **Runoff/percolation:** Water infiltration governed by temperature and snow density

Alpine3D Output Grids:

Alpine3D produces daily raster outputs (GeoTIFF + ASCII format):

- **HS (snow height, m):** Spatial distribution of accumulated snow depth
- **SWE (snow water equivalent, kg/m^2):** Total water content accounting for density
- **TSG (surface temperature, $^{\circ}\text{C}$):** Important for melt forecasting and avalanche trigger potential [6]
- **Incoming radiation grids:** Spatially explicit solar and longwave forcing

```
SMET 1.1 ASCII
[HEADER]
station_id = ZER2
station_name = Zermatt
longitude = 7.752
latitude = 46.029
altitude = 1640
[DATA]
Timestamp TA RH VW DW ISWR TSG ...
2026-04-02T12:00:00 -5.2 65.4 3.1 NW
150.0 -8.1 ...
```

Listing 1. Example of a SMET file

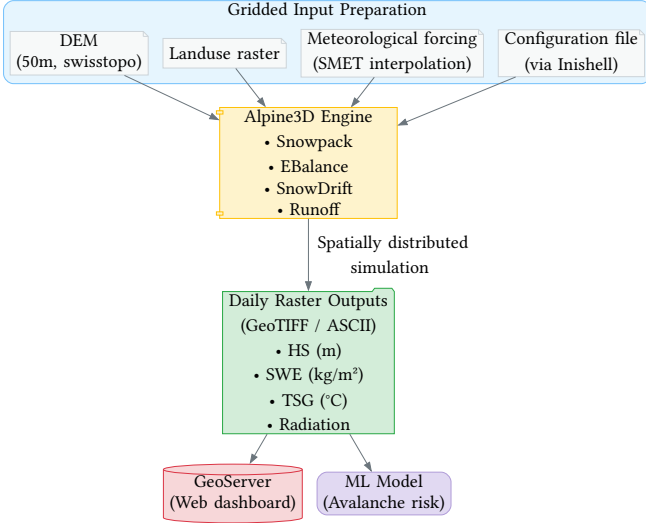


Fig. 4. Alpine3D service workflow

These grids are published to GeoServer for web dashboard visualization and are used to compute a grid of avalanche risk/danger using the ML model (Fig. 4).

c) Snowpack Model and Stratigraphy Analysis:

Snowpack Overview:

For measurement stations, we run the Snowpack model in 1D vertical mode to simulate internal snowpack stratigraphy and stability [7]. To accurately simulate snow conditions, the model uses point-scale meteorological forcing from SMET files and determines the snow surface's energy balance using meteorological interpolation. These inputs are then used to drive detailed parametrizations of snow metamorphism, consolidation, and layering [6].

Snowpack Feature Extraction: Snowpack outputs a .pro (profile) file containing the detailed snowpack state at each timestep (Fig. 5):

Stratigraphy features (layer-by-layer):

- Layer thickness, density, temperature, liquid water content
- Grain type and size evolution
- Hardness profile (coded numerically, 1–4 scale)

Aggregate stability indices (computed from profile) [6]:

$$\text{Skier Stability Index (SK38)} = \min_{\text{layers}} \frac{\tau_{\max}}{\sigma_n} \quad (2)$$

where $\tau_{\max}$ is shear strength and σ_n is normal stress, computed for each layer.

$$\text{Natural Stability Index}_{(\text{SN38})} = \text{SK38}_{\text{with natural load}} \quad (3)$$

$$\text{Critical Cut Length}_{(\text{CCL})} = \text{Length}_{\text{column slab release}} \quad (4)$$

- **Penetration resistance:** Simulates hard-layer detection
- **Wind transport indicators:** Cumulative wind speed x density parameter
- **Temperature gradient:** Identifies temperature-gradient metamorphism (TG) zones

These variables on the snow profile are then used to assess the avalanche danger level.

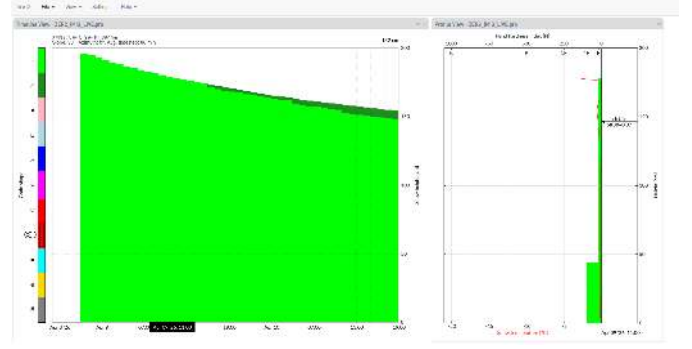


Fig. 5. Profile visualization in NiViz

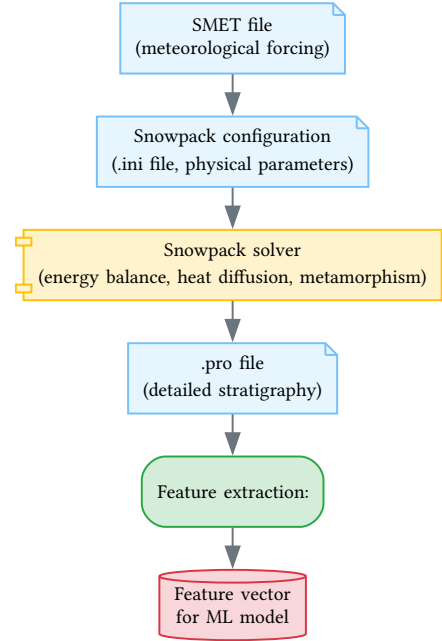


Fig. 6. Risk Prediction service pipeline

Snowpack Input/Output Workflow (Fig. 6):

Snowpack Predictive Variables:

We extract features from Snowpack for the ML risk model. Table III is a table of everything that is extracted.

d) Risk Model: Neural Network-Based Danger Level Prediction:

Model Architecture and Training Dataset:

A neural network model [8] is trained to map a feature vector (from Snowpack, Alpine3D and IMIS) to a discrete avalanche danger level:

$$\text{Danger Level} \in \{1, 2, 3, 4, 5\} \quad (5)$$

The danger level corresponds to:

- **1 (Low):** Avalanches unlikely
- **2 (Moderate):** Avalanches possible with high trigger probability, small natural avalanches
- **3 (Considerable):** Avalanches probable even with moderate triggers, large natural avalanches
- **4 (High):** Avalanches very likely, large to very large natural avalanches

TABLE III : NEURAL NETWORK FEATURES

Feature Group	Example Variables	Physical Meaning
New Snow	HN24, HN72_24, HN7d	Cumulative precipitation; higher values → increased avalanche probability
Layer Weakness	sk38_pwl (potential weak layer), sn38_pwl	Lower values indicate unstable layers prone to human/natural triggering
Deformation Stability	Sd, wind transport 3-day	Shear instability, high values from recent wind loading or rapid heating
Energy State	LWR_net, Qw (water content)	Net radiation, liquid water weakens bonds, promotes failure
Temperature Profile	TS1, TS2 (internal temps)	Weak layer formation in cold layers, surface crusts from solar input

- **5 (Very High):** Widespread natural avalanche activity, massive runs

Network architecture:

The model is a Feed-Forward Neural Network (Multilayer Perceptron) built using TensorFlow/Keras. It is designed for multi-class classification, predicting one of five avalanche danger levels.

Input Layer: The network takes an input vector of 38 features (combining core meteorological data, snowpack stability indices, and memory features like recent snow sums). These inputs are preprocessed using median imputation for missing values and standard scaling.

Training Process:

- **Data Split:** The data [9] is split into 80% training (23,436 samples) and 20% testing/validation (5,860 samples).
- **Optimization & Loss:** The model is compiled using the Adam optimizer with a learning rate of 0.001. Because the target labels are one-hot encoded, the loss function is set to Categorical Cross-entropy.
- **Training Loop:** The model trains for 100 epochs feeding data in batch sizes of 64.

North American Public Avalanche Danger Scale			
Avalanche danger is determined by the likelihood, size, and distribution of avalanches. Safe backcountry travel requires training and experience. You control your risk by choosing when, where, and how you travel.			
Danger Level	Travel Advice	Likelihood	Size and Distribution
5 - Extreme	Extraordinarily dangerous avalanche conditions. Avoid all avalanche terrain.	Nature and human triggered avalanches certain.	Very large avalanches in many areas.
4 - High	Very dangerous avalanche conditions. Travel in avalanche terrain not recommended.	Natural avalanches likely; human-triggered avalanches very likely.	Large avalanches in many areas; or very large avalanches in specific areas.
3 - Considerable	Dangerous avalanche conditions. Overall snowpack evaluation, cautious route finding, and conservative decision-making essential.	Natural avalanches possible; human-triggered avalanches likely.	Small avalanches in many areas; or large avalanches in specific areas; or very large avalanches in isolated areas.
2 - Moderate	Heightened avalanche conditions on specific terrain features. Evaluate snow and terrain carefully; identify features of concern.	Natural avalanches unlikely; human-triggered avalanches possible.	Small avalanches in specific areas; or large avalanches in isolated areas.
1 - Low	Generally safe avalanche conditions. Watch for unstable snow on isolated terrain features.	Natural and human-triggered avalanches unlikely.	Small avalanches in isolated areas or extreme terrain.

Fig. 7. North America Public Avalanche Danger Scale

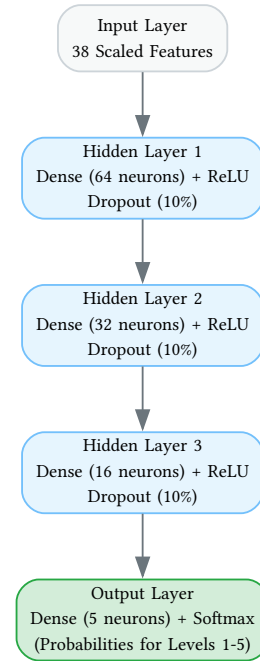


Fig. 8. Neural Network Architecture

- **Performance Reality:** While the model achieves a respectable 75% overall accuracy, the classification report reveals a stark class imbalance issue. It predicts levels 1, 2, and 3 reasonably well, but completely fails to generalize on the rare extreme events (Fig. 10).

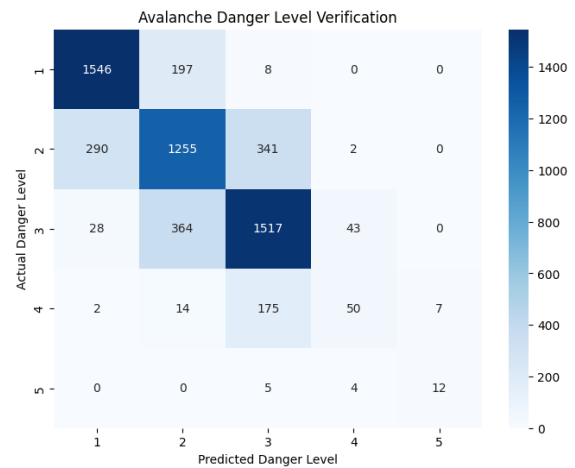


Fig. 9. Model confusion matrix

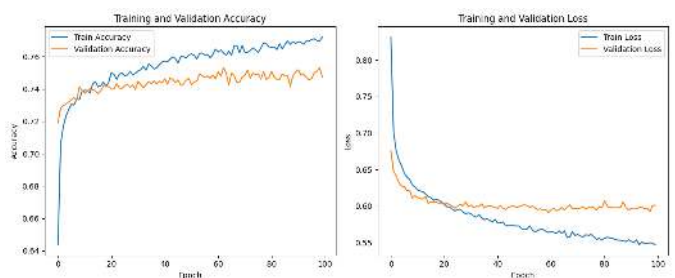


Fig. 10. Training Loss and Accuracy

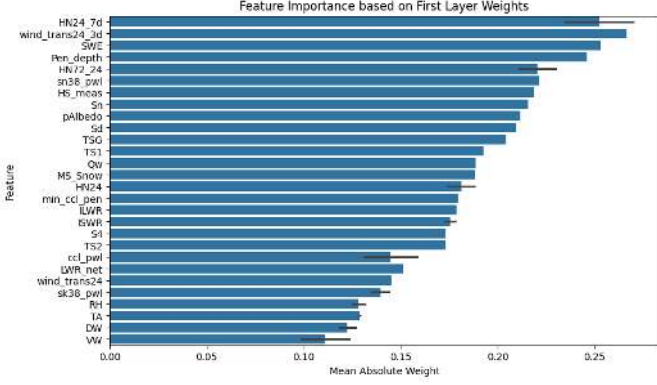


Fig. 11. Feature importance based on first layer

Feature Importance and Physical Interpretation:

The features importance are assessed from the mean absolute weights of the neural network's first hidden layer (Fig. 11). It reveals which physical parameters are the primary drivers of the model's avalanche danger predictions. Overall we observe that the model heavily prioritizes “memory” features (accumulated conditions).

Cumulative New Snow

Cumulative new snow variables HN24_7d (7-day sum), HN72_24 (3-day sum), and HN24 (24-hour sum) are strong predictors. This is also confirmed with high weight values given to SWE (Snow Water Equivalent) and Pen_depth (penetration depth) which are directly linked physically to cumulative new snow [10]

Wind transport Dynamics

3-day wind transport feature (wind_trans24_3d) is second. This is due to the fact that elevated wind speeds over multiple days physically transport and deposit snow and therefore creates dangerous weak layers conditions that the model recognizes as a trigger for elevated danger levels

Snowpack Stability Indices

Stratigraphy and stability indices generated by Alpine3D/ Snowpack, in particular sn38_pwl (natural stability) and sk38_pwl (skier stability), also carry significant weight in the network. Lower stability scores indicate higher risk.

Temperature and Energy State

The model relies on thermal and energy dynamics, represented in the graph by features like TSG (ground/surface temperature), TS1 and TS2 (internal snow temperatures), and TA (air temperature). These variables allow the network to capture critical thermal shifts: rapid warming events or spikes in liquid water content (Qw) are recognized as factors that quickly increase danger by weakening snow bonds, whereas stable, prolonged cold periods reduce the predicted risk.

B. Avalanche Simulation Service

a) Overview:

The simulation service generates avalanche propagation scenarios from input data such as terrain and release conditions. It acts as a core component of the digital twin

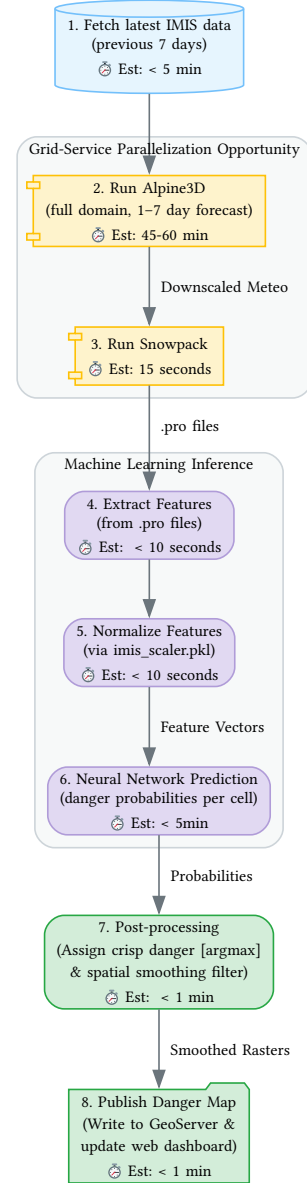


Fig. 12. Real-time prediction pipeline

by transforming environmental data into dynamic outputs suitable for visualization and analysis.

The service is structured as a three-stage pipeline: ingestion, AvaFrame simulation, and digestion, as shown in Fig. 13.

The ingestion stage prepares and formats the input data, the AvaFrame stage performs the physical simulation, and the

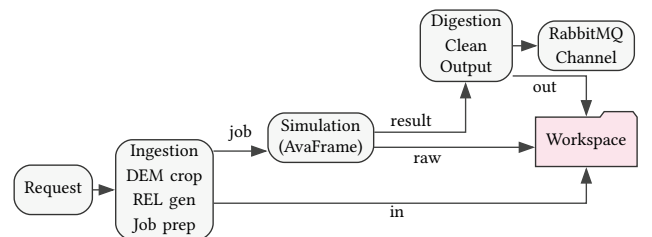


Fig. 13. Avalanche simulation workflow.

digestion stage processes the raw outputs into visualization-ready data.

This architecture ensures adaptability under AvaFrame’s strict interface constraints while enabling efficient parallel simulations through fine-grained resource allocation, without introducing unnecessary computational overhead.

b) Ingestion Phase:

The ingestion phase ensures compatibility between external data sources and the AvaFrame framework by standardizing and preparing inputs. It supports multiple input modes while maintaining a consistent and reproducible simulation setup.

Two ingestion workflows are implemented. The first relies on a pre-defined release file to directly generate simulations from existing inputs. The second operates from a geographic position, automatically retrieving the corresponding terrain data (DEM) and generating the release area (REL) based on user-defined parameters such as release radius and thickness.

This phase also acts as the entry point for simulation parameters provided by the prediction service, including snow height, snow density, and avalanche type (dry or wet), enabling dynamic configuration based on real-time or predicted conditions.

c) AvaFrame:

AvaFrame [11] is a Python framework developed by the Austrian Service for Torrent and Avalanche Control and the Austrian Research Centre for Forests. Its main objective is to accurately simulate both dry and wet avalanches.

Dry avalanches are mainly composed of snow powder and typically generate a large cloud of suspended particles, whereas wet avalanches are characterized by a denser and more compact snow flow, as illustrated in Figure 14. The avalanche type is primarily influenced by terrain, wind, temperature and snow conditions [12]. Therefore, a framework such as AvaFrame is essential to ensure that the digital twin can continuously simulate avalanche behavior under varying environmental conditions.

AvaFrame requires a set of structured inputs to perform avalanche simulations. The main inputs include:

- A DEM, representing the terrain
- A release area (REL), defining the avalanche starting zone
- Snow and simulation parameters, such as density and release thickness
- A configuration defining the simulation type and resolution



Fig. 14a. Dry avalanche



Fig. 14b. Wet avalanche

Fig. 14. Comparison of dry and wet avalanche simulations.

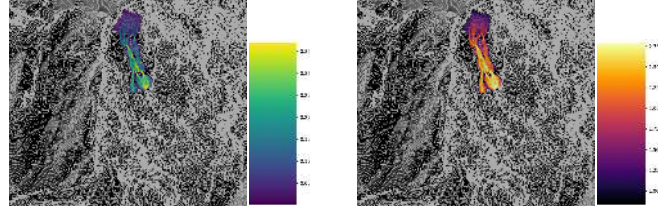


Fig. 16a. Flow thickness

Fig. 16b. Flow velocity

Fig. 16. Avalanche simulation outputs produced with AvaFrame. Flow thickness is shown in meters (linear scale), while flow velocity is displayed in meters per second (logarithmic scale).

These inputs must follow specific formatting and spatial constraints. The ingestion service generates the required files, such as DEM and REL rasters, and organizes them into a job-specific workspace, ensuring full compatibility with AvaFrame.

Simulation complexity depends on the size of the avalanche, with larger events requiring longer computation times. Computation ranges from a few minutes for small avalanches to several tens of minutes for larger ones. To reduce computation time, the number of simulated particles can be decreased while maintaining acceptable accuracy.

AvaFrame produces several outputs; however, the most relevant for our application are flow thickness and flow velocity over time. The spatial distribution of these outputs is shown in Figure 16.

d) Digestion Phase:

Similar to the ingestion phase, the digestion phase acts as a post-processing step that filters the outputs produced by AvaFrame, retaining only the relevant information and forwarding it to other services through communication channels or queues.

In particular, this phase prepares the simulation outputs for visualization by selecting the relevant frames generated by AvaFrame and converting them into a more suitable format. This process also reduces data volume and ensures consistency with visualization requirements.

C. Visualization Service

The visualization stack of AvalanCH serves as an interface for the user to simulate avalanches, see predicted risk zones, and snow elevation over a 3D map of the Zermatt region. Four components are at its core : The avalanche simulation shader and rendering logic, the fetching and layering of the predicted avalanche danger zones, the imagery and elevation data of Zermatt, and the user interface.

Initially, we wanted to use CesiumJS as our rendering engine, as bringing Swiss terrain and imagery data into it was very straightforward. However, adding 3D meshes, like the simulation of an avalanche was much more complex. As such, we opted for Godot, thanks to its ease of use and flexibility. As it is primarily aimed for video game development, we had full control over what and how we wanted to implement the visualization.

a) *Simulation rendering:*

To render the volumetric flow of snow of the avalanche, we use a raymarching algorithm in a fragment shader.

First, the maps coming from the simulation are extracted : flow thickness and flow velocity are used. Using the DEM, the absolute snow height is retrieved. Then, using a sequence of three compute shaders performing a Jump Flood Algorithm (JFA), we generate a Signed Distance Field (SDF). This SDF is then raymarched in local space to generate the snow volume on the fly. This allows for high scalability as there is no mesh generation bottleneck.

The lightweight snow that flies around the avalanche would benefit from a proper fluid simulation, but because of the tight frame budget we simply derive its density from the flow velocity. We use a standard cloud raytracing algorithm to render it.

Both types of snow are rendered using the same versatile shader, which allows for easy lighting, shadows and interactions.

b) *Risk prediction map layer:* As mentioned in the previous section, the risk prediction service generates a danger map depicting the risk level of avalanche everywhere in the Zermatt region. This map is published on a GeoServer, and a MapProxy service is used to make the data available to the Godot scene, which then projects the danger data over the 3D map, as seen in Fig. 17.

c) *Imagery and elevation data:* Much work was done to bring imagery and elevation data into Godot, in order to find a solution that's resource optimized, but still with satisfactory visual fidelity. As mentioned above, the initial solution was to get swiss topography data through Cesium, and simply render it in CesiumJS. Once the switch to Godot had been decided, multiple solutions were considered. First, a Cesium for Godot plugin was available, so we tested it as it was the simplest way of porting our work with Cesium into Godot. This solution was unfortunately flawed, as the scale of the 3D map was extremely small. Fetching the swissAlti3D data and rendering it manually also proved unfruitful, since the resources used were way too high for a single 1x1km tile.



Fig. 17. Danger map layer applied on the 3D map visualization

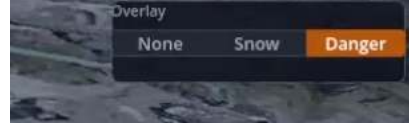


Fig. 18. Layer selection UI

Our final solution uses GeoServer to store topography, imagery and danger zone data. This tool, also known as a tile server, allows us to store the entire Zermatt zone's geospatial data, and serve it in tiles, as the user pans and zooms on the 3D map. This process resembles closely to how other map-based software like Google Maps are implemented. The tiles can also have level of detail (LOD), which is very useful for example with the topography data, as we don't need to load all the high precision tiles in view of the camera.

Additionally, MapProxy is used to cache the tiles provided by GeoServer, and make them available to Godot with minimal latency. A Godot script fetches these tiles, with a LOD logic depending on how close the camera is from the tile.

The benefit of this architecture is that any form of geospatial data can be loaded in the GeoServer, and we simply need to tweak the Godot script logic to display it.

d) *User interface:*

Our philosophy for the user interface was simplicity. The 3D map is controlled in a very similar manner as Google Earth or other 3D map software. The user can pan, zoom and tilt the camera just by using mouse controls. They can also toggle between the snowfall depth view, the avalanche danger view, and the normal view, with the buttons showed in Fig. 18.

Additionally, the avalanche simulation view has playback controls, as seen in Fig. 19. This allows for frame by frame control of the simulation.

IV. INTEGRATION

The work of connecting the different parts of AvalanCH together, like the avalanche prediction service, the simulation service and the visualization was something we greatly underestimated. We initially believed that, since we had an idea of the data formats required between each connection, adapting the services to receive and convert said data formats would be fairly simple. However, on multiple occasions, case-



Fig. 19. Playback control UI for avalanche simulation

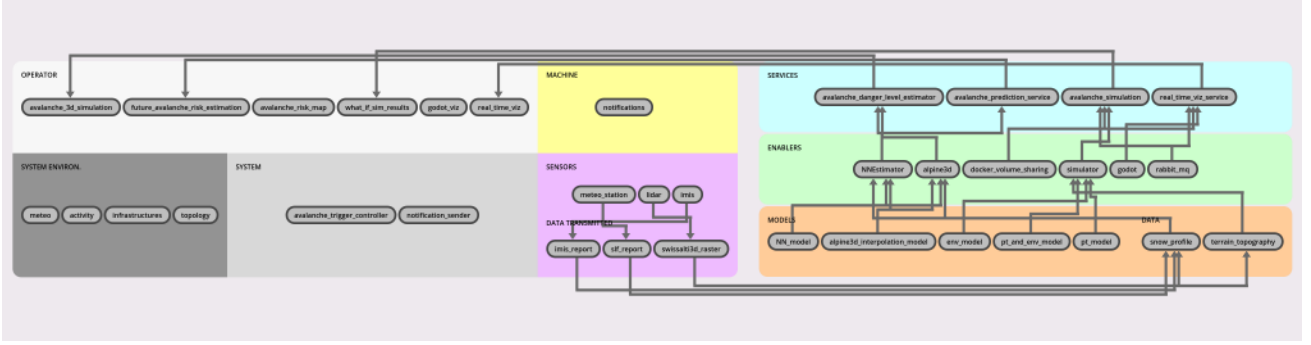


Fig. 20. AvalanCH DT Constellation, generated using DTInsight

specific challenges emerged, which hindered our ability to have a fully operational DT.

A. Prediction service and visualization

The integration of avalanche danger levels and snow height was relatively easy thanks to Geoserver. Geoserver enabled us to transform any grid (DEM, Geotiff) into any format. Since avalanche danger level and snow height were already grids. We just had to change the coordinates to match those of Geoserver and figure out a way to map the data we wanted to transfer (danger level and snow height). We decided to map those to the color using the gray scale and the styler. It was very similar to what has been done for the visualization of the topography of the mountains.

B. Simulation service and visualization

This integration was key to the design process of the visualization service : because a lot of data needs to be streamed at a very fast pace (16ms per frame for 60 frames per second), we knew from the very beginning that it would be a major bottleneck. Most of the heavy lifting is done in the visualization service. The integration itself is a shared docker volume between the two containers.

V. DESCRIPTION FRAMEWORK

On the subject of the framework characteristics, we've used the DTInsight tool to assess how exactly our work is compliant with the framework. This tool was used to generate the AvalanCH digital twin constellation, representing all services, components, their interactions and the characteristics Table in Appendix. Further explanation has been added for each characteristic.

VI. CHALLENGES AND FUTURE WORK

A. Challenges

a) Integration of undocumented or evolving open-source tools:

A key constraint of this project was the choice to rely on open-source software to ensure accessibility, reproducibility,

and ease of deployment. However, many of the selected tools, such as AvaFrame and Alpine3D, are primarily developed in research contexts and are not always well documented for non-expert users.

In practice, this led to several integration challenges. Documentation is often incomplete or outdated, and some software releases are no longer compatible with current data sources. For example, changes in IMIS data formats since the Alpine3D 2021 release prevented direct use of the model, requiring additional preprocessing to ensure compatibility with the other services. In addition, it was necessary to rebuild the software from source, compile it manually, and resolve several issues before obtaining a functional setup.

Several alternatives exist, such as RAMMS or r.avaflow [13] for avalanche simulation, and game engines such as Unity for visualization. However, these options introduce trade-offs in terms of accessibility, flexibility, or integration.

Overall, this highlights the challenges of balancing openness, usability, and system compatibility when working with scientific software.

b) Handling large-scale simulation data:

Another challenge of this project is the management of large-scale data. The DEM of the Zermatt region is approximately 12 GB, making it impractical to use directly within the visualization engine. Even for simulation purposes, it was necessary to split the dataset to improve performance. However, partitioning the DEM is not trivial, as the full extent of an avalanche must be preserved to avoid truncating the simulation.

More generally, the volume of data available through Swiss open data APIs is significant and requires careful handling. At the same time, obtaining consistent data across the entire Alpine region remains difficult. The limited number of sensors in mountainous areas requires combining multiple data sources and interpolation techniques to obtain usable inputs.

Data heterogeneity also introduces temporal challenges. Different sources provide data at varying timesteps and update frequencies, making it difficult to achieve near real-time updates, which are essential for time-critical applications such as rescue operations.

In addition, handling multiple coordinate systems and spatial resolutions adds further complexity. Data is provided

in different formats and reference systems, including geographic coordinates (latitude/longitude), Swiss projected coordinates (LV95), and engine-specific coordinate systems used in visualization tools. Ensuring consistency between these representations requires careful transformations and alignment.

Resolution requirements also vary significantly across components. High-resolution DEMs (e.g., 2 m) are required for realistic visualization, while coarser resolutions are used for simulation (e.g., 5 m for AvaFrame and up to 50 m for Alpine3D) to maintain reasonable computation times. This necessitates resampling and introduces additional sources of approximation.

Finally, differences in data formats must be addressed. While geospatial data is typically handled as GeoTIFF files, AvaFrame requires specific constraints such as the absence of NoData values, and the visualization engine does not support GeoTIFF directly, requiring conversion to formats such as OpenEXR. These transformations add complexity to the data pipeline and increase processing overhead.

Overall, this highlights the challenges of balancing data resolution, consistency, and real-time processing in environmental digital twins.

c) *Complexity of large-scale digital twin modeling:*

Modeling a digital twin of a large mountainous region is inherently challenging due to the scale of the data and the complexity of the environment. The size of the area to be modeled introduces significant computational and data management constraints, while the limited availability of sensor data requires combining multiple sources and approximation techniques.

In addition, most services in the pipeline are computationally intensive and operate asynchronously, as simulations and data processing steps can take several minutes to complete. This makes synchronization between components more difficult and complicates the design of a consistent and responsive system.

Overall, building such a system requires a substantial amount of software engineering to address the multiple challenges related to data handling, service orchestration, and system integration.

These challenges highlight several directions for future improvements of the system

B. Future Work

a) *Integration of real-time avalanche detection:*

One of the current limitations of the system is the lack of near real-time avalanche detection. Due to the limited number of sensors in mountainous areas, detection relies on sparse and indirect data sources.

A promising approach is the use of ski resort webcam networks combined with deep learning models such as YOLOv5. However, this raises challenges related to privacy and data access, as camera footage requires explicit authorization.

Several studies demonstrate the potential of such methods. Fox et al. [14] used ResNet152 and YOLOv3 to achieve high detection performance, while Liu et al. [15] improved results using more recent YOLO architectures with near real-time inference.

These approaches remain sensitive to environmental conditions such as weather, visibility, and camera placement [14]. Alternative solutions include satellite-based detection using Sentinel-1 SAR data [16], providing large-scale coverage with revisit times of a few days.

Other sensing modalities can also be considered, including seismic sensors [17], UAV-based monitoring [18], roadside radar [19], and infrasound sensors [20].

Achieving near real-time detection therefore relies on combining multiple sensing modalities, improving robustness and enabling more reliable monitoring of avalanche activity.

b) *Extension toward multi-scale or multi-region digital twins:*

Our approach has one main advantage : it is scalable at multiple stages. First, the data processing pipeline involving Alpine 3D can be scaled by allocating more resources, as it is already done by the SLF for example. Then, the simulation can be scaled by spawning multiple containers for simulating individual avalanches. Finally, the visualization already manages the whole Swiss Alps.

Our approach is also well-suited for a “DT of DTs” approach. We did some testing regarding segmenting mountains based on flow maps, and it is promising. If the mountains are segmented, the problem becomes trivially parallelizable (it is actually an Embarrassingly Parallel problem) at all the stages.

c) *Digital Twin companion application:*

A potential extension of this work is the development of a mobile application to inform and alert skiers about avalanche risks in real time, helping to prevent accidents.

A key feature would be the possibility for users to share their location (subject to privacy regulations such as the GDPR), enabling the system to better assess exposure to risk and detect when individuals are in hazardous areas or affected by an avalanche.

This would also provide valuable feedback to the physical system, for example by triggering targeted monitoring actions such as deploying drones to affected areas. Such capabilities could improve situational awareness, support real-time tracking, and reduce response times for rescue teams.

C. Online resources

The source code is located at <https://github.com/KevinPeymani/AvalanCH-DT>

The DT report is located at <https://afymer.github.io/avalanchDT-report/>

A short video explaining how AvalanCH works can be found at <https://www.youtube.com/watch?v=TzLxHhmUyXw>

APPENDIX

TABLE IV : DIGITAL TWIN CHARACTERISTICS

ID	Characteristic	Component	Description
C1	System under study		Avalanche digital twin for monitoring, prediction, and simulation of avalanche risks in the Swiss Alps.
C2	Physical acting components	avalanche_trigger_controller notification_sender	avalanche_trigger_controller: Controls avalanche trigger grid; notification_sender: Sends notifications to users. This has not been implemented yet. However, sending phone notifications is not something new, and should be quite straightforward to integrate to AvalanCH.
C3	Physical sensing components	meteo_station; lidar; imis	We did not take part in the development of the physical weather and snow sampling stations. We simply access their data through APIs of the Swiss government. We still think they fit in this characteristic because the weather data they provide is concrete, and we could, in theory, measure it ourselves.
C4	Physical-to-virtual interaction	imis_report; swissalti3d_raster slf_report;	The weather, snow and topography data are packed in standardized formats, which our different services have been designed to be compatible with from the get-go.
C5	Virtual-to-physical interaction		As mentioned in C2, this has not been implemented.
C6	DT services	avalanche_danger_level_estimator; avalanche_prediction_service; avalanche_simulation; real_time_viz_service	Please read Section II for more details.
C7	Twinning time-scale		Some aspects, like the weather and snow profile data need days to change significantly. Others, like the notifications being sent to the users operate on a much smaller time scale.
C8	Multiplicities		Please read Section VI.B on the extension of AvalanCH to multi-scale/ multi-region digital twins.
C9	Life-cycle stages		Our DT does not represent a manufactured or engineered system, so the only life-cycle stage we can assign it to is service.
C10	DT models and data	Models: NN_model; alpine3d_interpolation_model; env_model; pt_model; Data: snow_profile; terrain_topography	See the architecture section for more detail on each model. Some models, like the terrain topography have been used without any alterations.
C11	Tooling and enablers	NNEstimator; alpine3d; docker_volume_sharing; simulator; godot; rabbitmq	We use NNEstimator in order to generate an avalanche danger map from the results of alpine3d, AvaFrame is our simulator for avalanche simulation, godot enables the visualization and RabbitMq enables our services to broadcast signals that data is ready in shared Docker volumes.
C12	DT constellation		Please see Fig. 20 for the constellation.
C13	Twinning process and evolution		The DT was engineered based on flexible requirements, with a Minimal Viable Product consisting of only the avalanche danger prediction. This requires the full pipeline (from data acquisition) to visualization to be functional.
C14	Fidelity and validity		Not yet validated against historical data
C15	DT technical connection		The PT-to-DT connection is a bit tricky to define in our case, because all of our data technically comes from the PT, but at very different rates. Immutable data (the DEM for example) comes from the Alps but won't change until a few hundred years, while weather data has a granularity of a few hours. Most data however is drained over the Ethernet using RESTful APIs, and a few files (the DEM for example) are statically stored with the DT.
C16	DT hosting/deployment		As all the components of AvalanCH have been containerized, the deployment of the entire platform can be done easily on any cloud provider, or on a private server depending on the user's needs. The only element that the user needs deployed locally is the Godot executable, or AvalanCH "game".

ID	Characteristic	Component	Description
C17	Insights and decision making	3D simulation; risk estimation; risk maps; what-if analysis; visualization; notifications	Multiple features of AvalanCH have been added to aid the user in decision making, through avalanche simulation, risk analysis, and so on. Insight and decision making is at the core of our digital twin.
C18	Horizontal integration		There is horizontal integration with the services of the Swiss Alps DT. The DT is able to exchange information with other information systems via volume sharing and message passing using RabbitMQ.
C19	Data ownership and privacy		Datasets are the exclusive property of SLF and other governmental organisations. No privacy-related data is stored by the DT.
C20	Standardization		Communication are carried out using RabbitMQ, as well as standard WMTS tiles format, WMS and GeoTIFF.
C21	Security and safety	TLS encryption possible	Communication can be TLS encrypted through the RabbitMQ broker. No impactfull or potentially harmful action is taken by the DT for the time being, except sending notifications to users.

REFERENCES

- [1] European Commission, “Destination Earth—Shaping Europe’s Digital Future,” 2023.
- [2] W. Kritzing, M. Karner, G. Traar, J. Henjes, and W. Sihn, “Digital Twin in manufacturing: A categorical literature review and classification,” *Ifac-PapersOnline*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [3] I. Measurement and I. S. IMIS, “IMIS measuring network.” [Online]. Available: <https://www.envodat.ch/dataset/imis-measuring-network>
- [4] M. Lehning, I. Völksch, D. Gustafsson, T. A. Nguyen, M. Stähli, and M. Zappa, “ALPINE3D: a detailed model of mountain surface processes and its application to snow hydrology,” *Hydrological Processes: An International Journal*, vol. 20, no. 10, pp. 2111–2128, 2006.
- [5] M. Bavay, M. Reisecker, T. Egger, and D. Korhammer, “Inishell 2.0: semantically driven automatic GUI generation for scientific models,” *Geoscientific Model Development*, vol. 15, no. 2, pp. 365–378, 2022, doi: 10.5194/gmd-15-365-2022.
- [6] P. Bartelt and M. Lehning, “A physical SNOWPACK model for the Swiss avalanche warning: Part I: numerical model,” *Cold Regions Science and Technology*, vol. 35, no. 3, pp. 123–145, 2002, doi: [https://doi.org/10.1016/S0165-232X\(02\)00074-5](https://doi.org/10.1016/S0165-232X(02)00074-5).
- [7] *Looking back at the last 15 years of operational avalanche warning with the SNOWPACK model in Switzerland*. Zenodo, 2024. doi: 10.5281/zenodo.13861453.
- [8] C. Pérez-Guillén *et al.*, “Data-driven automated predictions of the avalanche danger level for dry-snow conditions in Switzerland,” *Natural Hazards and Earth System Sciences*, vol. 22, no. 6, pp. 2031–2056, 2022, doi: 10.5194/nhess-22-2031-2022.
- [9] C. Pérez-Guillén *et al.*, “Weather, snowpack and danger ratings data for automated avalanche danger level predictions.” [Online]. Available: https://www.envodat.ch/dataset/weather-snowpack-danger_ratings-data
- [10] F. Techel, B. Zweifel, and K. Winkler, “Analysis of avalanche risk factors in backcountry terrain based on usage frequency and accident data in Switzerland,” *Natural Hazards and Earth System Sciences*, vol. 15, no. 9, pp. 1985–1997, 2015, doi: 10.5194/nhess-15-1985-2015.
- [11] F. Oesterle, A. Wirbel, J.-T. Fischer, A. Huber, and P. Spannring, “OpenNHM/AvaFrame: 2.0rc3.” [Online]. Available: <https://doi.org/10.5281/zenodo.19096590>
- [12] J. Schweizer, J. Bruce Jamieson, and M. Schneebeli, “Snow avalanche formation,” *Reviews of Geophysics*, vol. 41, no. 4, p. , 2003, doi: <https://doi.org/10.1029/2002RG000123>.
- [13] M. Mergili, J.-T. Fischer, J. Krenn, and S. P. Pudasaini, “r. avaflow v1, an advanced open-source computational framework for the propagation and interaction of two-phase mass flows,” *Geoscientific Model Development*, vol. 10, no. 2, pp. 553–569, 2017.
- [14] J. Fox, A. Siebenbrunner, S. Reitering, D. Peer, and A. Rodríguez-Sánchez, “Automating avalanche detection in ground-based photographs with deep learning,” *Cold Regions Science and Technology*, vol. 223, p. 104179, 2024.
- [15] Z. Liu, X. Zhu, L. Pang, X. Fu, H. Zhu, and X. Liu, “AVA-YOLO: Image-based multiscale feature fusion enhanced perception model for snow avalanche detection,” *Measurement Science and Technology*, vol. 35, no. 12, p. 125804, 2024.
- [16] M. Eckerstorfer, H. Vickers, E. Malnes, and J. Grahm, “Near-real time automatic snow avalanche activity monitoring system using Sentinel-1 SAR data in Norway,” *Remote Sensing*, vol. 11, no. 23, p. 2863, 2019.
- [17] A. Van Herwijnen and J. Schweizer, “Monitoring avalanche activity using a seismic sensor,” *Cold Regions Science and Technology*, vol. 69, no. 2–3, pp. 165–176, 2011.
- [18] M. B. Bejiga, A. Zeggada, and F. Melgani, “Convolutional neural networks for near real-time object detection from UAV imagery in avalanche search and rescue operations,” in *2016 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, 2016, pp. 693–696.
- [19] L. Meier, M. Jacquemart, B. Blattmann, and B. Arnold, “Real-time avalanche detection with long-range, wide-angle radars for road safety in Zermatt, Switzerland,” in *Proceedings of the International Snow Science Workshop, Breckenridge, CO*, 2016, pp. 304–308.
- [20] E. Marchetti, M. Ripepe, G. Ulivieri, and A. Kogelnig, “Infrasound array criteria for automatic detection and front velocity estimation of snow avalanches: towards a real-time early-warning system,” *Natural*

Hazards and Earth System Sciences, vol. 15, no. 11, pp. 2545–2555, 2015.